

Ekino SEO Crawler

Documentation Technique

Version 1.0

Auteurs

Axel Rosa & Oriane Foy

PHP 8.2 · Laravel 12 · MySQL 8 · Symfony DomCrawler · Chart.js

2026

Table des matières

Table des matières	2
1. Présentation du projet	4
1.1 Objectifs	4
1.2 Technologies	4
2. Architecture du projet	5
2.1 Couches de l'application	5
2.2 Flux de traitement	5
2.3 Routes HTTP	6
3. Structure des fichiers	7
3.1 Arborescence	7
3.2 Index des fichiers	8
4. Documentation des classes PHP	9
4.1 Controllers	9
RequestUrlController	9
DashboardController	9
PageController	9
4.2 Models	9
URL	9
Constantes	9
Attributs	10
Site	10
4.3 Interface Constants	11
4.4 Services	12
CrawlService	12
ParseSiteService	12
4.5 Services de crawl	13
UrlPersistenceService	13
4.6 Services SEO	14
SeoExtractor	14
SeoAnalyzer	15
4.7 TagAssignerService	16
4.8 CrawlUrlJob	17
5. Vues Blade	18
request.blade.php	18
dashboard.blade.php	18
details.blade.php	18
sidebar.blade.php	19
6. Fichiers JavaScript	20
Tableau.js	20
Configuration et état	20
Pipeline de filtrage	20
Badges colorés	20
Détection des problèmes SEO (tooltip)	20

darkmode.js	21
SquareCharts.js	21
chart.js	22
chartSeoError.js	22
7. Infrastructure Docker et Makefile	24
7.1 Dockerfile	24
7.2 docker-compose.yml	24
7.3 Makefile	24
8. Base de données	26
8.1 Table sites	26
8.2 Table urls	26
9. Guide d'installation	27
9.1 Sans Docker	27
9.2 Avec Docker	27
9.3 Configuration .env	27
9.4 Lancer l'application	27
10. Cas d'utilisation	29
10.1 Lancer un crawl	29
10.2 Consulter le dashboard	29
10.3 Détail d'une URL	29
11. Améliorations possibles	30
11.1 Architecture	30
11.2 Fonctionnalités	30
11.3 Qualité du code	30
11.4 Infrastructure	30

1. Présentation du projet

Ekino SEO Crawler est une application web Laravel 12 permettant d'analyser automatiquement les pages d'un site via son sitemap XML. Pour chaque URL découverte, l'outil effectue une requête HTTP, extrait les données SEO (title, meta, H1-H4, canonical, robots, schema.org) et génère un rapport consultable via un tableau de bord interactif.

1.1 Objectifs

- Découverte automatique des URLs via sitemap.xml (récursif)
- Analyse SEO de chaque page : classification OK / Warning / Error
- Tableau de bord avec graphiques, filtres, tri, pagination
- Page de détail par URL : métriques HTTP + problèmes SEO
- Tagging automatique des URLs (homepage, article, product, contact...)

1.2 Technologies

Technologie	Version	Rôle
PHP	8.2+	Langage principal
Laravel	12.x	Framework MVC, routing, queue, Eloquent ORM
MySQL	8.x	Base de données relationnelle
Symfony DomCrawler	^8.0	Parsing HTML et extraction SEO
Chart.js	4.4.0	Graphiques front-end (CDN)
TailwindCSS	CDN	Framework CSS utilitaire
Docker / docker-compose	—	Conteneurisation (app + MySQL + phpMyAdmin)
Makefile	—	Automatisation des commandes projet

2. Architecture du projet

Le projet suit le pattern MVC de Laravel, enrichi d'une couche Service pour la logique métier et d'une couche Job pour le traitement asynchrone.

2.1 Couches de l'application

Couche	Dossier	Rôle
Controller	App\Http\Controllers	Réception HTTP, validation, orchestration
Service	App\Services	Toute la logique métier
Job	App\Jobs	Tâches asynchrones via Laravel Queue
Model	App\Models	Entités Eloquent (URL, Site)
Interface	App\Interfaces	Constantes globales partagées
View	resources/views	Templates Blade + TailwindCSS
JS Front	public/js	Tableau interactif, graphiques, dark mode
Docker	Dockerfile + docker-compose.yml	Infrastructure conteneurisée
Makefile	Makefile	Commandes raccourcies (infra, queue)

2.2 Flux de traitement

Pipeline entièrement asynchrone via Laravel Queue :

```
POST /site
├─ RequestUrlController.store()
│   └─ CrawlService.crawl()
│       └─ FetchSitemapJob [queue: default]
│           └─ GetSiteUrlService.__invoke() → ParseSiteService (récuratif)
│               └─ Pour chaque URL → CrawlUrlJob [queue: crawl]
│                   └─ UrlCrawlGuard → skip si crawlé < 5 min
│                       └─ HttpClient → GET HTTP + timer
│                           └─ SeoExtractor → parse HTML
│                               └─ SeoAnalyzer → scoring SEO
│                                   └─ UrlPersistenceService → save HTTP
│                                       └─ SeoPersistenceService → save SEO
│                                           └─ TagAssignerService → tag URL
│                                               └─ URL::save() → commit BDD
```

2.3 Routes HTTP

Méthode	URI	Contrôleur	Action
GET	/	—	Formulaire de saisie (request.blade.php)
POST	/site	RequestUrlController	Lance le crawl
GET	/dashboard	DashboardController	Tableau de bord
GET	/details/{id}	PageController	Détail d'une URL

3. Structure des fichiers

Liens GitHub cliquables

Tous les noms de fichiers ci-dessous sont des hyperliens. Remplace GITHUB_REPO_URL par ton URL de depot via Ctrl+H dans Word.

3.1 Arborescence

```
├── app/
│   ├── Http/Controllers/
│   │   ├── DashboardController.php
│   │   ├── PageController.php
│   │   └── RequestUrlController.php
│   ├── Jobs/
│   │   ├── CrawlUrlJob.php
│   │   └── FetchSitemapJob.php
│   ├── Models/ URL.php / Site.php
│   ├── Services/
│   │   ├── Crawl/ HttpCrawlerClient.php UrlCrawlGuard.php UrlPersistenceService.php
│   │   ├── Seo/ SeoAnalyzer.php SeoExtractor.php SeoPersistenceService.php
│   │   ├── Tag/ TagAssignerService.php
│   │   ├── CrawlService.php GetSiteUrlService.php ParseSiteService.php
│   │   ├── TableauDataService.php ChartsDataService.php
│   └── Interfaces/Constants.php
├── database/migrations/
├── public/js/
│   ├── Tableau.js SquareCharts.js chart.js chartSeoError.js darkmode.js
├── resources/views/
│   ├── dashboard.blade.php details.blade.php
│   ├── request.blade.php sidebar.blade.php
├── routes/web.php
├── Dockerfile docker-compose.yml Makefile
└── composer.json
```

3.2 Index des fichiers

Controllers

- [DashboardController.php](#)
- [RequestUrlController.php](#)
- [PageController.php](#)

Models

- [URL.php](#)
- [Site.php](#)

Services

- [CrawlService.php](#)
- [GetSiteUrlService.php](#)
- [ParseSiteService.php](#)
- [TableauDataService.php](#)
- [ChartsDataService.php](#)
- [Crawl/HttpCrawlerClient.php](#)
- [Crawl/UrlCrawlGuard.php](#)
- [Crawl/UrlPersistenceService.php](#)
- [Seo/SeoExtractor.php](#)
- [Seo/SeoAnalyzer.php](#)
- [Seo/SeoPersistenceService.php](#)
- [Tag/TagAssignerService.php](#)

Jobs

- [FetchSitemapJob.php](#)
- [CrawlUrlJob.php](#)

Vues Blade

- [request.blade.php](#)
- [dashboard.blade.php](#)
- [details.blade.php](#)
- [sidebar.blade.php](#)

JavaScript

- [Tableau.js](#)
- [SquareCharts.js](#)
- [chart.js](#)
- [chartSeoError.js](#)
- [darkmode.js](#)

Infrastructure

- [Dockerfile](#)
- [docker-compose.yml](#)
- [Makefile](#)
- [routes/web.php](#)

4. Documentation des classes PHP

4.1 Controllers

RequestUrlController

Fichier : <app/Http/Controllers/RequestUrlController.php>

Reçoit le formulaire POST, valide l'URL et délègue au CrawlService.

Paramètre	Type	Description
\$request	Request	Requête HTTP entrante
\$crawlService	CrawlService	Injecté par Laravel (injection de méthode)

Retour : RedirectResponse avec message flash de succès

DashboardController

Fichier : <app/Http/Controllers/DashboardController.php>

Orchestre les données du tableau de bord. Lit site_id (défaut 1), récupère les 3 jeux de données et les passe à la vue.

Attribut injecté	Type	Rôle
\$tableauDataService	TableauDataService	Données du tableau principal
\$chartsDataService	ChartsDataService	Données des graphiques et compteurs

PageController

Fichier : <app/Http/Controllers/PageController.php>

Charge une URL par son ID et affiche sa page de détail. Lance une 404 automatique si l'ID n'existe pas (findOrFail).

4.2 Models

URL

Fichier : <app/Models/URL.php>

Modèle Eloquent central. Représente une URL crawlée. Chaque setter met à jour l'attribut privé ET le tableau \$attributes d'Eloquent pour garantir la persistance.

Constantes

Constante	Valeur	Sémantique
SEO_STATUS_OK	'ok'	Aucun problème SEO
SEO_STATUS_WARNING	'warning'	Avertissements SEO

Constante	Valeur	Sémantique
SEO_STATUS_ERROR	'error'	Erreurs SEO bloquantes
STATUS_COMPLETED	'completed'	Crawl réussi
STATUS_FAILED	'failed'	Crawl en erreur

Attributs

Attribut PHP	Colonne BDD	Type	Description
\$url	url	string	URL complète de la page
\$statusCode	status_code	?int	Code HTTP (200, 301, 404...)
\$responseTime	response_time	?int	Temps de réponse en ms
\$responseSize	response_size	?float	Taille de la réponse (octets)
\$responseCacheHit	response_cache_hit	?bool	Cache HTTP détecté
\$responseRedirection	response_redirection	?string	URL de redirection
\$seoTitle	seo_title	?string	Contenu de la balise <title>
\$seoH1Title	seo_h1_title	?string	Texte du premier H1
\$seoMetaDescription	seo_meta_description	?string	Meta description
\$seoCanonical	seo_canonical	?string	URL canonique déclarée
\$seoMetaRobots	seo_meta_robots	?string	Valeur meta robots
\$seoH1Number...\$seoH4Number	seo_h1_number...seo_h4_number	?int	Nombre de balises Hn
\$seoStatus	seo_status	?string	ok / warning / error
\$seoFixes	seo_fixes (JSON)	?array	Liste des problèmes SEO
\$seoSchema	seo_schema (JSON)	?array	Données JSON-LD schema.org
\$tags	tags (JSON)	array	Tags de catégorisation
\$country	country	?string	Code pays extrait du chemin

Site

Fichier : [app/Models/Site.php](#)

Entité représentant un site. Créée automatiquement via firstOrCreate() lors du premier crawl d'un domaine. Attribut fillable : name (domaine).

4.3 Interface Constants

Fichier : <app/Interfaces/Constants.php>

Centralise toutes les constantes partagées entre les services.

```
<?php
namespace App\Interfaces;

interface Constants
{
    // User-Agent envoyé dans chaque requête HTTP
    public const USER_AGENT = 'Ekino-SEO-Crawler (Stage FOY Oriane, ROSA Axel)';

    // XPath – URLs dans un sitemap standard
    public const URLQUERY = '//sm:url/sm:loc';

    // XPath – Détection d'un sitemap index (sitemap de sitemaps)
    public const QUERYPATH = '//sm:sitemap/sm:loc | //sitemap/loc';

    public const TIMEOUT_SECONDS = 45;
    public const URL_SOURCE_CRAWL = 'crawl';
    public const URL_SOURCE_SITEMAP = 'sitemap';

    // Headers HTTP indiquant la présence d'un cache
    public const CACHE_HEADERS = ['X-Cache', 'CF-Cache-Status', 'Age'];
}
```

4.4 Services

CrawlService

Fichier : [app/Services/CrawlService.php](#)

Point d'entrée du crawl. Identifie ou crée le Site, puis dispatche FetchSitemapJob sur la queue 'default'.

```
public function crawl(string $startUrl, int $maxUrls = 0): void
{
    $baseDomain = parse_url($startUrl, PHP_URL_HOST);
    $site = Site::firstOrCreate(['name' => $baseDomain]);
    FetchSitemapJob::dispatch($startUrl, $site, $maxUrls);
}
```

ParseSiteService

Fichier : [app/Services/ParseSiteService.php](#)

Parsing XML récursif. Gère les sitemaps index (contenant d'autres sitemaps) et les sitemaps standard. Filtre les URLs hors domaine via parse_url().

```
public function parseSitemap(string $xml, string $baseHost, string $baseDomain): array
{
    $dom = new DOMDocument();
    $xpath = new DOMXPath($dom);
    $xpath->registerNamespace('sm', 'http://www.sitemaps.org/schemas/sitemap/0.9');

    // Est-ce un sitemap index ? (contient des <sitemap> enfants)
    $sitemapNodes = $xpath->query(Constants::QUERYPATH);
    if ($sitemapNodes && $sitemapNodes->length > 0) {
        // Appel récursif sur chaque sous-sitemap
        foreach ($sitemapNodes as $node) {
            $response = Http::get(trim($node->nodeValue));
            $subUrls = $this->parseSitemap($response->body(), $baseHost, $baseDomain);
            $urls = array_merge($urls, $subUrls);
        }
    } else {
        // Sitemap standard : extraction des <loc> filtrés par domaine
        $urlNodes = $xpath->query(Constants::URLQUERY);
        foreach ($urlNodes as $node) {
            $url = trim($node->nodeValue);
            $host = parse_url($url, PHP_URL_HOST);
            if ($host === $baseDomain) { $urls[] = $url; }
        }
    }
    return $urls;
}
```

4.5 Services de crawl

UrlPersistenceService

Fichier : [app/Services/Crawl/UrlPersistenceService.php](#)

Service invocable (`__invoke`). Crée ou met à jour le modèle URL avec les métriques HTTP. Contient deux logiques commentées ci-dessous.

```
public function __invoke(
    string $url, int $siteId, $response,
    float $responseTime, bool $usingSitemap = false
): URL {
    $urlModel = URL::firstOrCreate(['url' => $url, 'site_id' => $siteId]);

    // — Extraction du code pays depuis le chemin de l'URL —————
    // Ex : /fr-FR/page → segments[0]='fr-FR' → langexplode[1]='FR'
    $path          = parse_url($url)['path'] ?? '';
    $segments      = explode('/', trim($path, '/'));
    $langParts     = explode('-', $segments[0] ?? '');
    $urlModel->setCountry(trim($langParts[1] ?? ''));

    // — Métriques HTTP —————
    $urlModel->setStatusCode($response->status());
    $urlModel->setResponseTime((int) round($responseTime * 1000)); // s → ms
    $urlModel->setResponseSize((int) round(strlen($response->body())));
    $urlModel->setResponseContentType($response->header('Content-Type'));

    // — Détection de redirection —————
    if ($response->status() >= 300 && $response->status() < 400) {
        $urlModel->setResponseRedirection($response->header('Location'));
    }

    // — Détection cache (X-Cache, CF-Cache-Status, Age) —————
    $urlModel->setResponseCacheHit(
        collect(Constants::CACHE_HEADERS)->some(
            fn($h) => $response->header($h) !== null
        )
    );

    $urlModel->save();
    return $urlModel;
}
```

4.6 Services SEO

SeoExtractor

Fichier : <app/Services/Seo/SeoExtractor.php>

Extraction des données SEO depuis le HTML brut via Symfony DomCrawler. Retourne un tableau associatif avec title, meta_description, h1, h1_count...h4_count, canonical, robots, schemas (JSON-LD), meta_keywords.

```
public function extract(string $htmlContent): array
{
    $crawler = new Crawler($htmlContent);

    // Extraction des éléments textuels standards
    $title     = $crawler->filter('title')->count()
                ? $crawler->filter('title')->first()->text() : null;

    $metaDesc  = $crawler->filter('meta[name="description"]')->count()
                ? $crawler->filter('meta[name="description"]')->attr('content') : null;

    $h1        = $crawler->filter('h1')->count()
                ? $crawler->filter('h1')->first()->text() : null;

    $h1Count   = $crawler->filter('h1')->count();
    $h2Count   = $crawler->filter('h2')->count();
    $h3Count   = $crawler->filter('h3')->count();
    $h4Count   = $crawler->filter('h4')->count();

    $canonical = $crawler->filter('link[rel="canonical"]')->count()
                ? $crawler->filter('link[rel="canonical"]')->attr('href') : null;

    // Extraction des blocs JSON-LD schema.org
    $schemas = [];
    foreach ($crawler->filter('script[type="application/ld+json"]') as $script) {
        $schemas[] = json_decode($script->nodeValue, true);
    }

    return [
        'title'           => $title,
        'meta_description' => $metaDesc,
        'h1'              => $h1,
        'h1_count'        => $h1Count,
        'h2_count'        => $h2Count,
        'h3_count'        => $h3Count,
        'h4_count'        => $h4Count,
        'canonical'       => $canonical,
        'schemas'         => $schemas,
    ];
}
```

SeoAnalyzer

Fichier : <app/Services/Seo/SeoAnalyzer.php>

Applique les règles SEO sur les données extraites. Détermine le statut global : error si au moins une erreur, warning si seulement des avertissements, ok sinon.

```
private const TITLE_MIN = 30; private const TITLE_MAX = 60;
private const DESC_MIN = 120; private const DESC_MAX = 160;

public function analyze(array $seoData): array
{
    $fixes = []; $hasError = false;

    // Règle 1 – Title : obligatoire, longueur 30-60 car.
    if (!$seoData['title']) {
        $fixes[] = ['code' => 'title_missing', 'severity' => 'error',
            'message' => 'Title tag is missing'];
        $hasError = true;
    } elseif (strlen($seoData['title']) < self::TITLE_MIN
        || strlen($seoData['title']) > self::TITLE_MAX) {
        $fixes[] = ['code' => 'title_length', 'severity' => 'warning',
            'message' => 'Title hors plage 30-60'];
    }

    // Règle 2 – H1 : obligatoire ET unique
    if (!$seoData['h1']) {
        $fixes[] = ['code' => 'h1_missing', 'severity' => 'error',
            'message' => 'H1 tag is missing'];
        $hasError = true;
    } elseif ($seoData['h1_count'] > 1) {
        $fixes[] = ['code' => 'multiple_h1', 'severity' => 'warning',
            'message' => "Multiple H1 ({$seoData['h1_count']})"];
    }

    // Règle 3 – Meta description : recommandée, longueur 120-160 car.
    if (!$seoData['meta_description']) {
        $fixes[] = ['code' => 'meta_missing', 'severity' => 'warning',
            'message' => 'Meta description is missing'];
    } elseif (strlen($seoData['meta_description']) < self::DESC_MIN
        || strlen($seoData['meta_description']) > self::DESC_MAX) {
        $fixes[] = ['code' => 'meta_length', 'severity' => 'warning',
            'message' => 'Meta description hors plage 120-160'];
    }

    return [
        'status' => $hasError ? 'error' : (empty($fixes) ? 'ok' : 'warning'),
        'fixes' => $fixes
    ];
}
```

4.7 TagAssignerService

Fichier : [app/Services/Tag/TagAssignerService.php](#)

Catégorise les URLs par analyse du path. Le premier pattern reconnu gagne. Défaut : tag 'page'.

```
private const PATTERNS = [
    'homepage' => ['/', ''],
    'article' => ['/article', '/articles', '/blog', '/post', '/news'],
    'product' => ['/product', '/produit'],
    'category' => ['/category', '/categorie'],
    'faq' => ['/faq', '/help', '/aide'],
    'contact' => ['/contact'],
    'about' => ['/about', '/a-propos'],
    'legal' => ['/legal', '/privacy', '/mentions-legales', '/cgv', '/cgu'],
    'auth' => ['/login', '/register', '/connexion', '/inscription'],
];

public function assignTags(URL $urlModel): void
{
    $path = strtolower(parse_url($urlModel->getUrl(), PHP_URL_PATH) ?? '/');

    if ($path === '/' || $path === '') {
        $urlModel->setTags(['homepage']); return;
    }
    foreach (self::PATTERNS as $tag => $patterns) {
        if ($tag === 'homepage') continue;
        foreach ($patterns as $pattern) {
            if (str_contains($path, $pattern)) {
                $urlModel->setTags([$tag]); return;
            }
        }
    }
    $urlModel->setTags(['page']); // Aucun pattern reconnu → défaut
}
```


4.8 CrawlUrlJob

Fichier : <app/Jobs/CrawlUrlJob.php>

Job central (queue: crawl). Orchestre le pipeline complet pour une URL. Les dépendances sont injectées par le container Laravel dans la méthode handle().

```
public function handle(
    UrlCrawlGuard      $guard,
    HttpCrawlerClient  $crawler,
    UrlPersistenceService $persistence,
    SeoExtractor        $seoExtractor,
    SeoAnalyzer         $seoAnalyzer,
    TagAssignerService $tagAssigner,
    SeoPersistenceService $seoPersistence,
): void {
    // Étape 1 – Déduplication : skip si crawlé < 5 min
    if ($guard->hasBeenCrawledRecently($this->url, $this->site->id)) {
        return;
    }

    // Étape 2 – Requête HTTP avec mesure du temps
    $result = $crawler->crawl($this->url);

    // Étape 3 – Extraction et scoring SEO
    $seoData      = $seoExtractor->extract($result['response']->body());
    $seoAnalysis  = $seoAnalyzer->analyze($seoData);

    // Étape 4 – Persistance HTTP
    $urlModel = $persistence(
        url:          $this->url,
        siteId:       $this->site->id,
        response:     $result['response'],
        responseTime: $result['response_time'],
        usingSitemap: $this->usingSitemap
    );

    // Étape 5 – Persistance SEO + tag + sauvegarde finale
    $seoPersistence->saveSeoResults($urlModel, $seoData, $seoAnalysis);
    $tagAssigner->assignTags($urlModel);
    $urlModel->save();
}
```

5. Vues Blade

Les vues utilisent TailwindCSS (CDN) avec support dark mode via la classe CSS 'dark'. La sidebar est incluse dans toutes les pages via `@include('sidebar')`.

request.blade.php

Fichier : <resources/views/request.blade.php>

Page d'accueil. Formulaire de saisie d'URL avec validation côté serveur et affichage du message flash.

Élément	Description
Formulaire	POST vers route('request-url.store') avec @csrf
Input url	required url — validation Laravel
Flash	@if(session('success')) — affichage conditionnel du message de retour

dashboard.blade.php

Fichier : <resources/views/dashboard.blade.php>

Tableau de bord principal. Graphiques Chart.js, 4 cartes de statistiques cliquables et tableau paginé.

Section	Description
Graphiques	seoErrorChart (erreurs HTTP+SEO) + seoSeverityChart (warning vs error)
Cartes cliquables	Multiple H1 / Pages lentes / Meta vide / Pages lourdes — déclenchent un filtre tableau
Filtres	Status HTTP, statut SEO, pays, recherche texte
Tableau	10 lignes/page, tri Ms et Visite, badges colorés, bouton Voir
Injection JS	window.seoErrorData et window.tableData via @json(\$variable)

details.blade.php

Fichier : <resources/views/details.blade.php>

Section	Données affichées
HTTP	Status code, temps, taille, cache, Content-Type, redirection
SEO	Badge ok/warning/error + liste des corrections seo_fixes
Contenu	Titre, meta description, schema.org JSON-LD

sidebar.blade.php

Fichier : [resources/views/sidebar.blade.php](#)

Sidebar de navigation commune. Liens vers / et /dashboard avec icônes SVG et support dark mode TailwindCSS.

6. Fichiers JavaScript

Tableau.js

Fichier : <public/js/Tableau.js>

Module principal de gestion du tableau interactif. Expose `window.TableauApp`. Fonctionne sur `window.tableData` injecté par Blade.

Configuration et état

```
const CONFIG = {
  perPage: 10, // Lignes affichées par page
  slowPageThreshold: 600, // ms – seuil page lente
  largeSizeThreshold: 2 * 1024 * 1024 // 2 Mo – seuil page lourde
};

const state = {
  currentPage: 1,
  cardFilter: null, // Filtre actif depuis les cartes du dashboard
  sort: { ms: null, visite: null }
};

// Données injectées par Blade
let tableData = Array.isArray(window.tableData)
  ? window.tableData : Object.values(window.tableData || {});
```

Pipeline de filtrage

```
function getFiltered() {
  let result = tableData;
  result = applyCardFilter(result); // Filtre carte cliquée
  result = applySearchFilter(result); // Recherche texte libre
  result = applyStatusFilter(result); // Code HTTP / statut SEO
  result = applyCountryFilter(result); // Code pays
  result = applySort(result); // Tri colonne
  return result;
}
```

Badges colorés

```
function statusBadge(code) {
  const n = Number(code);
  let cls = 'bg-gray-100 text-gray-800';
  if (n === 200) cls = 'bg-green-100 text-green-800';
  else if (n >= 300 && n < 400) cls = 'bg-blue-100 text-blue-800';
  else if (n === 404) cls = 'bg-yellow-100 text-yellow-800';
  else if (n >= 500) cls = 'bg-red-100 text-red-800';
  return `<span class='px-2 py-1 rounded text-xs font-medium ${cls}'>${code}</span>`;
}

function seoBadge(s) {
  const colors = { ok: 'bg-green-100 text-green-800',
    warning: 'bg-yellow-100 text-yellow-800',
    error: 'bg-red-100 text-red-800' };
  return `<span class='${colors[s] || ''}'>${s}</span>`;
}
```

Détection des problèmes SEO (tooltip)

```
function getSeoTooltipContent(row) {
  const issues = [];
  if (row.seoH1Number > 1)
    issues.push({ icon: 'warning', text: 'Multiple H1 tags' });
  if (row.seoH1Number === 0)
    issues.push({ icon: 'error', text: 'Missing H1 tag' });
  if (!row.seoTitle?.trim())
    issues.push({ icon: 'error', text: 'Missing title tag' });
  if (!row.seoMetaDesc?.trim())
    issues.push({ icon: 'warning', text: 'Missing meta description' });
  if (row.responseTime > CONFIG.slowPageThreshold)
    issues.push({ icon: 'warning', text: `Slow page (${row.responseTime}ms)` });
  if (row.pageSize > CONFIG.largeSizeThreshold)
    issues.push({ icon: 'warning', text: 'Large page size' });
  return issues.length ? issues : null;
}
```

darkmode.js

Fichier : <public/js/darkmode.js>

Gestion du mode sombre via la classe CSS 'dark'. Persistance dans localStorage.

```
function toggleDarkMode() {
  document.documentElement.classList.toggle('dark');
  localStorage.setItem('darkMode',
    document.documentElement.classList.contains('dark'));
}

// Au chargement : restaure le mode depuis localStorage
if (localStorage.getItem('darkMode') === 'false') {
  document.documentElement.classList.remove('dark');
}
```

SquareCharts.js

Fichier : <public/js/SquareCharts.js>

Gère les filtres déclenchés par les cartes cliquables. Expose window.applyCardFilter() et window.clearCardFilter().

```
// Appel : applyCardFilter('multiple_h1', 'Multiple H1')
window.applyCardFilter = function (filterType, filterLabel) {
  if (window.TableauApp?.setCardFilter) {
    TableauApp.setCardFilter(filterType); // Délègue à Tableau.js
  }
  document.querySelector('section.bg-white')
    ?.scrollIntoView({ behavior: 'smooth', block: 'start' });
  showFilterIndicator(filterLabel);
};

window.clearCardFilter = function () {
  TableauApp?.clearCardFilter();
  document.getElementById('cardFilterIndicator')?.remove();
};

function showFilterIndicator(label) {
  // Insère un bandeau bleu au-dessus du tableau
  const indicator = document.createElement('div');
  indicator.id = 'cardFilterIndicator';
```

```

indicator.className = 'mb-4 p-3 bg-blue-50 border border-blue-200 rounded-lg';
indicator.innerHTML = `
    <span>Filtre actif : <strong>${label}</strong></span>
    <button onclick='clearCardFilter()'>Effacer</button>`;
// Injection dans le DOM avant le tableau
const tableSection = document.querySelector('section.bg-white > div.mb-4');
tableSection?.parentNode.insertBefore(indicator, tableSection.nextSibling);
}

```

chart.js

Fichier : <public/js/chart.js>

Graphique en barres horizontales des sévérités SEO (Warning vs Error). Cible le canvas #seoSeverityChart. Alimenté par window.seoErrorData injecté par Blade.

```

(function () {
    if (!window.seoErrorData || !window.Chart) return;

    const warning = window.seoErrorData['SEO Warning'] || 0;
    const error = window.seoErrorData['SEO Error'] || 0;

    new Chart(document.getElementById('seoSeverityChart'), {
        type: 'bar',
        data: {
            labels: ['SEO Warning', 'SEO Error'],
            datasets: [{
                data: [warning, error],
                backgroundColor: ['#facc15', '#ef4444']
            }]
        },
        options: {
            indexAxis: 'y', // Barres horizontales
            plugins: { legend: { display: false } }
        }
    });
})();

```

chartSeoError.js

Fichier : <public/js/chartSeoError.js>

Graphique en barres verticales des erreurs HTTP et SEO agrégées (404, 301, Missing title...). Adapte l'axe Y au maximum des données.

```

(function () {
    if (!window.seoErrorData || !window.Chart) return;

    // Filtre les entrées SEO Warning/Error (gérées par chart.js)
    const entries = Object.entries(window.seoErrorData)
        .filter(([label]) => label !== 'SEO Warning' && label !== 'SEO Error');

    const labels = entries.map(([label]) => label);
    const data = entries.map([, value] => value);

    // Rouge pour les erreurs manquantes, orange pour les avertissements
    const colors = labels.map(label =>
        label.includes('Missing') || label.includes('No ')
        ? '#ef4444' : '#f59e0b'
    );
})();

```

```
// Axe Y adaptatif : +10% au-dessus du max
const maxValue      = Math.max(...data, 1);
const suggestedMax = Math.ceil(maxValue * 1.1) + 1;

new Chart(document.getElementById('seoErrorChart'), {
  type: 'bar',
  data: { labels, datasets: [{ data, backgroundColor: colors }] },
  options: { scales: { y: { beginAtZero: true, suggestedMax } } }
});
})();
```

7. Infrastructure Docker et Makefile

7.1 Dockerfile

Fichier : [Dockerfile](#)

```
FROM php:8.4-apache

RUN a2enmod rewrite
RUN apt-get update && apt-get install -y \
    git unzip libzip-dev libpng-dev libonig-dev libxml2-dev libicu-dev
RUN docker-php-ext-install pdo_mysql zip intl mbstring exif pcntl bcmath gd

COPY --from=composer:2 /usr/bin/composer /usr/bin/composer

# Redirige Apache vers le dossier public/ de Laravel
ENV APACHE_DOCUMENT_ROOT=/var/www/html/public
RUN sed -ri 's!/var/www/html!/var/www/html/public!g' \
    /etc/apache2/sites-available/*.conf

WORKDIR /var/www/html
```

7.2 docker-compose.yml

Fichier : [docker-compose.yml](#)

```
services:
  app:                # Laravel Apache – port 8000
    build: .
    ports: ['8000:80']
    volumes: ['./var/www/html']
    depends_on: [db]

  db:                 # MySQL 8 – port 3306
    image: mysql:8.0
    environment:
      MYSQL_DATABASE: laravel
      MYSQL_USER: laravel
      MYSQL_PASSWORD: laravel
      MYSQL_ROOT_PASSWORD: root
    volumes: [dbdata:/var/lib/mysql]

  phpmyadmin:         # Interface SQL – port 8080
    image: phpmyadmin/phpmyadmin
    environment: { PMA_HOST: db, PMA_USER: root, PMA_PASSWORD: root }
    ports: ['8080:80']

volumes:
  dbdata:
```

7.3 Makefile

Fichier : [Makefile](#)

Cible	Description
make infra-up	docker-compose up -d --build
make infra-stop	docker-compose down
make infra-bash	bash dans le container app
make queue-start	1 worker default + 20 workers crawl (parallèle)
make queue-stop	Arrête tous les workers
make queue-status	Workers actifs + jobs en attente
make queue-logs	tail -f storage/logs/laravel.log
make queue-failed	Liste les jobs échoués
make queue-retry	Relance tous les jobs échoués
make queue-flush	Supprime tous les jobs échoués

8. Base de données

8.1 Table sites

Colonne	Type	Contrainte	Description
id	bigint unsigned	PK auto-increment	Identifiant unique
name	varchar(255)	NOT NULL	Domaine (ex: example.com)
created_at / updated_at	timestamp	nullable	Dates auto Laravel

8.2 Table urls

Colonne	Type	Description
id	bigint PK	Identifiant
url	varchar UNIQUE	URL complète
status	varchar DEFAULT 'completed'	Statut du crawl
status_code	int nullable	Code HTTP
response_time	int nullable	Temps (ms)
response_size	float nullable	Taille (octets)
response_cache_hit	boolean nullable	Cache détecté
response_redirection	varchar nullable	URL de redirection
seo_title	varchar nullable	Balise <title>
seo_meta_description	text nullable	Meta description
seo_canonical	varchar nullable	URL canonique
seo_meta_robots	varchar nullable	Meta robots
seo_h1_number...seo_h4_number	int nullable	Nombre de balises Hn
seo_status	varchar nullable	ok / warning / error
seo_fixes	json nullable	Corrections SEO
seo_schema	json nullable	JSON-LD schema.org
tags	json nullable	Tags de catégorisation
country	varchar nullable	Code pays
site_id	bigint FK → sites	Site parent

9. Guide d'installation

9.1 Sans Docker

```
git clone <URL_DU_REPO> && cd <dossier>

# Installation automatique (script Composer)
composer run setup

# Ou étape par étape :
composer install
cp .env.example .env
php artisan key:generate
php artisan migrate --force
npm install && npm run build
```

9.2 Avec Docker

```
make infra-up
make infra-bash
> composer install
> cp .env.example .env # DB_HOST=db
> php artisan key:generate
> php artisan migrate
```

9.3 Configuration .env

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1 # 'db' avec Docker
DB_PORT=3306
DB_DATABASE=laravel
DB_USERNAME=root
DB_PASSWORD=

QUEUE_CONNECTION=database
```

9.4 Lancer l'application

```
# Développement (serveur + queue + logs + vite en parallèle)
composer run dev

# Avec Docker
make queue-start
```

Queue obligatoire

Sans un worker actif les jobs restent en attente indefiniment : aucune URL ne sera crawlee.

10. Cas d'utilisation

10.1 Lancer un crawl

#	Acteur	Action
1	Utilisateur	Ouvre / et saisit https://example.com
2	RequestUrlController	Validation required url
3	CrawlService	firstOrCreate(Site), dispatch FetchSitemapJob
4	FetchSitemapJob	Récupère sitemap.xml → dispatch N x CrawlUrlJob
5	CrawlUrlJob (xN)	HTTP → extract → analyze → persist → tag → save
6	Utilisateur	Rafraîchit /dashboard pour voir les résultats

10.2 Consulter le dashboard

#	Acteur	Action
1	Utilisateur	GET /dashboard?site_id=1
2	DashboardController	Récupère tableData + errorData + chartData
3	Vue + JS	Rendu du tableau filtrable et graphiques Chart.js
4	Utilisateur	Filtre, trie, clique sur une carte pour filtrage rapide

10.3 Détail d'une URL

#	Acteur	Action
1	Utilisateur	Clique sur Voir dans le tableau
2	PageController	URL::findOrFail(\$id)
3	Vue details	Métriques HTTP + statut SEO + corrections

11. Améliorations possibles

11.1 Architecture

- Couche Repository pour abstraire les DB::select() directs dans les services
- DTOs (Data Transfer Objects) plutôt que tableaux associatifs entre services
- Découpler le pipeline de CrawlUrlJob avec des événements Laravel

11.2 Fonctionnalités

- Crawl récursif des liens internes (pas uniquement le sitemap)
- Analyse des images : alt manquant, taille excessive
- Export CSV/PDF des résultats
- Planification de crawls récurrents (Laravel Scheduler)
- Alertes email lors de nouvelles erreurs SEO
- Comparaison entre deux crawls dans le temps

11.3 Qualité du code

- Corriger le parametre \$seoH1Number dans setSeoH2Number() — URL.php
- Tests unitaires pour SeoAnalyzer, SeoExtractor, TagAssignerService
- Extraire les seuils 600ms et 2 Mo du JS en configuration serveur
- Gestion des timeouts explicite dans HttpCrawlerClient

11.4 Infrastructure

- Redis pour la queue à la place de la base de données
- Laravel Horizon pour le monitoring des queues en temps réel
- CI/CD avec GitHub Actions pour les tests automatiques